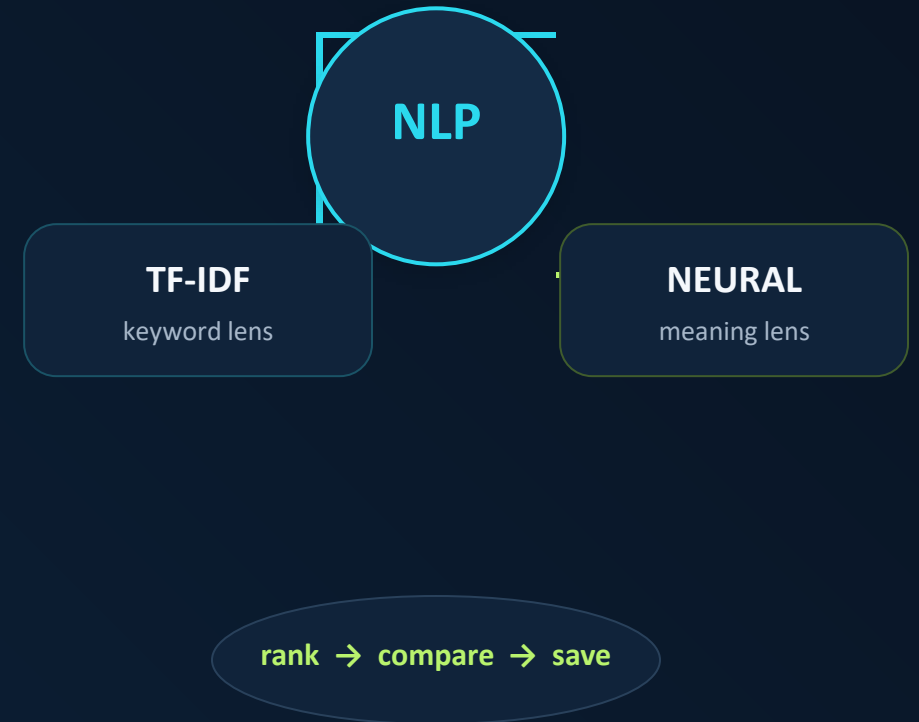


AI-POWERED COURSE RECOMMENDATION SYSTEM

Project overview: problem, models, architecture, and product flow.

Final Year Project

Group ID F25PROJECTEC4D8 | Roll No BC240419615
Supervisor Muhammad Bilal



The challenge is not a lack of courses — it is finding the right ones.

Manual browsing is slow, while exact keyword search can miss the learner's intent.

A learner may know the outcome they want, but not the exact course title or vocabulary used by the catalog.

SmartCourse starts with the learner's own words and turns them into a ranked, inspectable set of options.

LEARNER INPUT

**“I want to learn Python
for data science.”**

WHY SEARCH BREAKS

01 Vocabulary mismatch

The course may say “predictive modeling” while the learner says “data science.”

02 Catalog scale

Thousands of records make manual comparison expensive.

03 Trust and choice

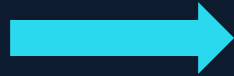
The learner needs relevance signals and a way to compare rankings.

SmartCourse turns one sentence into a learning path.

The product keeps ranking explainable: every result includes a course, a score, and enough context to judge the match.

NATURAL-LANGUAGE GOAL

learn Python
for data science



RANKING ENGINES

TF-IDF
precise keyword alignment

Neural
semantic similarity



RANKED OUTPUT

Course match

score 0.91

title · level · rating

The learner can inspect, compare, save, and revisit — not just receive a black-box answer.

That combination of relevance + traceability is the product's core value.

Two models answer two different questions.

TF-IDF asks “which courses use similar words?” Neural search asks “which courses express a similar meaning?”

KEYWORD LENS

TF-IDF

Best when exact terms matter

- spaCy cleans and lemmatizes the text
- Unigrams + bigrams build the vector space
- Cosine similarity ranks the catalog
- Fast, transparent, easy to explain

“Python”, “SQL”, “cloud”

MEANING LENS

NEURAL

Best when intent is implicit

- MiniLM converts text into embeddings
- Course embeddings are precomputed
- Query embedding is compared at runtime
- Related wording can still match

“predictive modeling” ≈ “machine learning”

Hybrid mode in this prototype = both ranked lists shown side by side.

The expensive work happens offline; the user query stays lightweight.

Cleaning and embedding the catalog creates reusable artifacts that the Flask app loads when needed.

OFFLINE PREPARATION



ONLINE QUERY

Load model → encode query → calculate similarity → return Top-N → persist session

Saved artifacts: `models/tfidf_recommender.joblib` and `models/neural_recommender.joblib`

Three tiers keep the system understandable.

The Flask application factory wires the UI, services, persistence, and model artifacts together.

PRESENTATION TIER

Bootstrap + Jinja pages

Home

Recommendations

Dashboard

About

APPLICATION TIER

Flask controllers + services

API blueprint

Recommend service

HistoryService

Logging

DATA / MODEL TIER

Persistent data + trained artifacts

SQLite

CSV catalog

TF-IDF artifact

Neural artifact

Cross-cutting concerns: configuration · validation · CORS · error handling · rotating logs

JSON endpoints keep browser and ranking logic loosely coupled.

This keeps the frontend replaceable and makes the recommendation workflow testable outside the UI.

POST /api/recommend

```
{
  "preference": "Python for data
science",
  "model": "hybrid",
  "top_n": 10
}
```

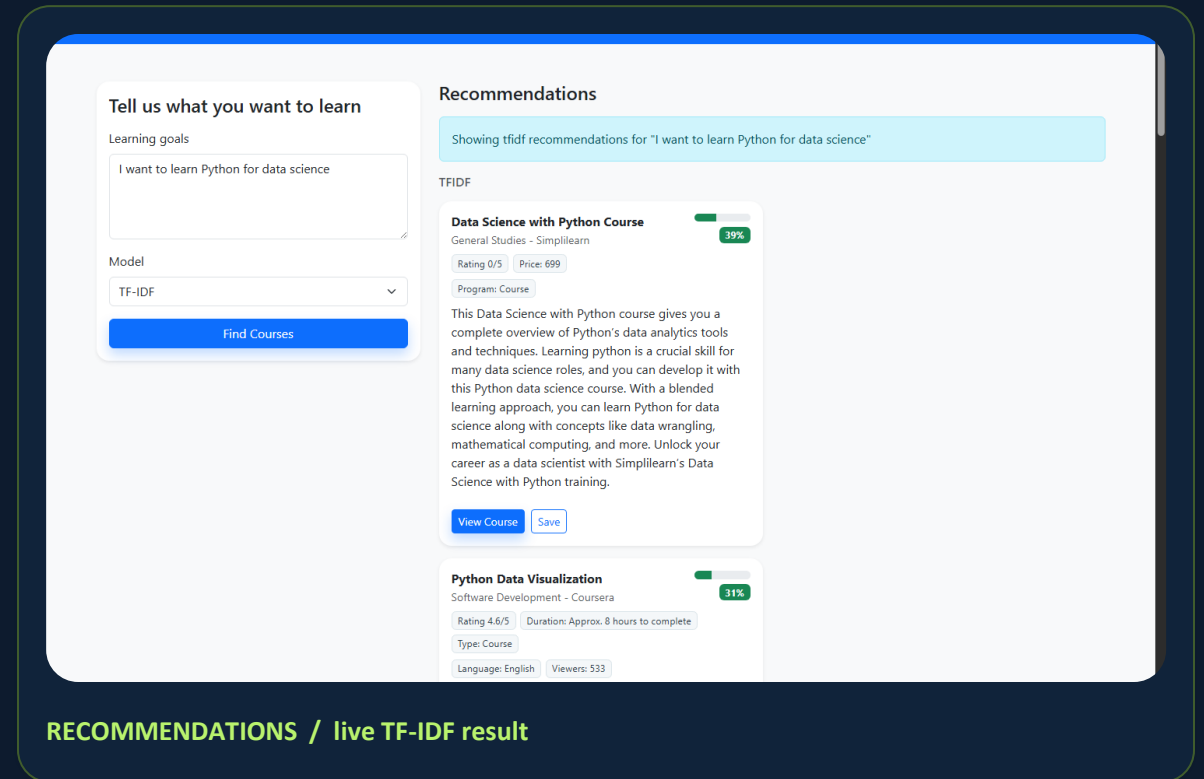
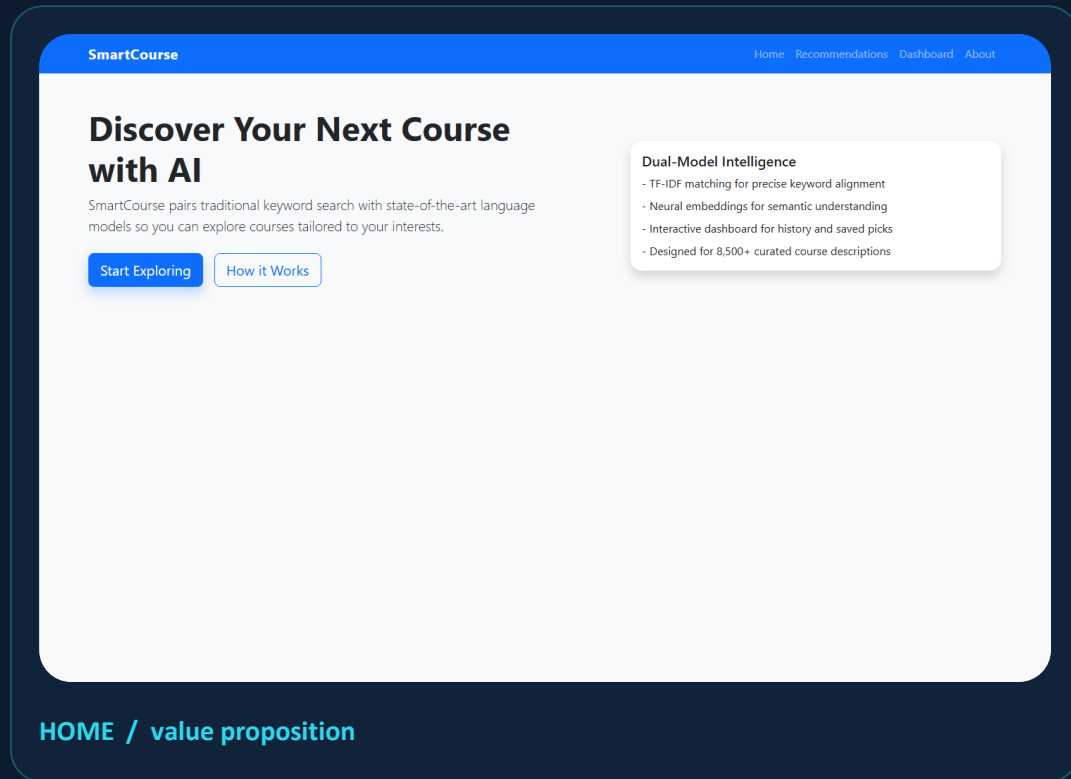
→ response: results + session_id

POST	/api/recommend	validate → rank → persist
GET	/api/history	history + saved results
POST	/api/save	bookmark a course
DELETE	/api/history	clear sessions
DELETE	/api/saved	clear bookmarks

Validation lives at the boundary; ranking and persistence live in services.

The running app makes the recommendation workflow visible.

These screenshots were captured from the local Flask application using the project's real Home and Recommendations pages.



The dashboard preserves the learner trail after a search.

The captured state shows search history, saved recommendations, and the model-comparison area working together.

The dashboard is divided into three main sections: Search History, Saved Recommendations, and Model Comparison.

Search History (with Refresh and Clear buttons):

- I want to learn Python for data science** (18/08/2026, 8:50:03 pm) **1 saved**
- I want to learn Python for data science** (18/08/2026, 8:49:34 pm) **0 saved**
- I want to learn Python** (29/04/2026, 5:44:41 am) **1 saved**

Saved Recommendations (with Clear Saved button):

- Data Science with Python Course** (39%)
General Studies - TFIDF
18/08/2026, 8:50:15 pm
Rating 0/5 | Price: 699 | Provider: Simplilearn
View Course
- Data Science with Python Course** (25%)
General Studies - TFIDF
29/04/2026, 5:44:51 am
Rating 0/5 | Price: 699 | Provider: Simplilearn
View Course

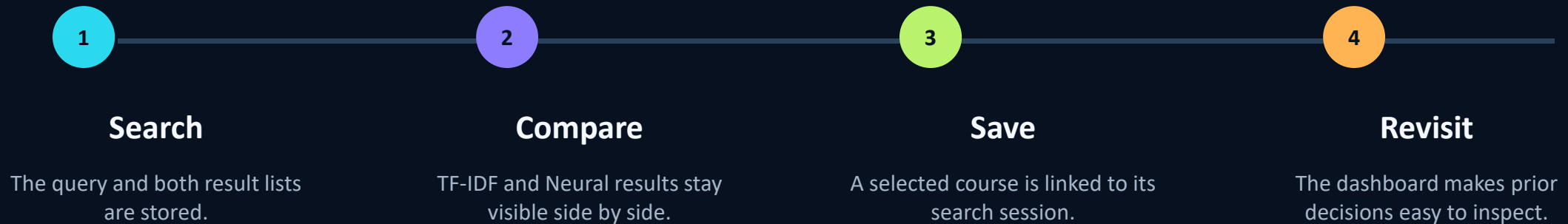
Model Comparison
Select a search session from history to compare TF-IDF and Neural results side by side.

TF-IDF

DASHBOARD / persistence + comparison

A one-time search becomes a reusable learning trail.

Every search becomes a session; promising courses can be saved and the two ranking perspectives can be compared later.



SQLite tables

search_sessions ↔ saved_recommendations

JSON keeps rich course metadata portable.

The project measures ranking quality — not just screen polish.

Evaluation tooling compares how useful the top of each ranking is for predefined queries and relevance labels.

Precision@K

Of the top K courses, how many are relevant?

Recall@K

Of all known relevant courses, how many appear in the top K?

Hit Rate

Did the top K contain at least one relevant course?

How the evaluation is run



Honest scope note: the checked evaluation set is small, so metrics are a starting point—not a claim of universal quality.

The technology choices reflect the prototype's goals and constraints.

Each component supports a clear, testable recommendation workflow.

Why TF-IDF?

It is fast, transparent, and strong when exact technical terms matter.

What is cosine similarity?

It compares vector direction; a higher value means a closer match in that representation.

What is Hybrid?

Both models run and the ranked lists are shown side by side; scores are not fused.

Why sentence transformers?

They capture semantic meaning, so related phrases can match without identical words.

Why Flask and SQLite?

Flask is lightweight for pages plus APIs; SQLite is file-based and enough for prototype persistence.

What are the limits?

Small evaluation data, no accounts or personalization yet, and prototype-level deployment.

FUTURE WORK

Feedback-based re-ranking · weighted score fusion · larger labels · filters · production security

The product flow is search → compare → save → revisit.

The demo connects the user journey to the engineering behind it.

01

Search

Open Recommendations
Enter: Python for data
science

UI → API

02

Compare

Select Hybrid
Explain both ranking lenses

models → cards

03

Save

Choose a course
Point out score and
metadata

POST /api/save

04

Revisit

Open Dashboard
Show history + saved item

GET /api/history

Request flow: validate → rank → normalize → persist → render.

THE TAKEAWAY

SmartCourse makes course discovery more explainable.

It combines two complementary ranking methods with a usable web experience, persistent learner history, and a clear path toward personalization.

Problem

large catalog

Solution

two ranking lenses

Outcome

better decisions

DISCUSSION

Questions and feedback